

Trylo.pk — Product & Technical Specification

For: Claude Code / Development Team Prepared from founder discovery sessions with Ahmad (founder, Lahore/Hafizabad, Pakistan) **Status:** Pre-build. A working interactive prototype exists (single-file HTML/JS, see Appendix A) and has been tested with real users. **This document specifies the REAL product to be engineered from scratch — the prototype is a UX/interaction reference only, not a codebase to extend.**

0. Read This First — Honest Scoping

This document is written to prevent two failure modes: (1) underbuilding something that feels like a toy, and (2) overpromising AI capabilities that don't actually exist as described. Specifically:

- **There is no such thing as "proportional pixel scaling" for virtual try-on.** Body-accurate garment try-on is a hard ML problem (trained diffusion / garment-warping models), not a geometry/math problem. Do not attempt to build this in-house for an MVP. Integrate a third-party try-on API (see Section 7).
- **Product photos must come from real onboarded sellers who own the rights to them.** Do not scrape, screenshot, or otherwise source images from the internet, competitor sites, or social media. This is a legal requirement, not a style preference.
- **"Memory" and "taste personalization" are achievable as a rules-based recommendation engine for MVP.** True deep-learning personalization (the TikTok/Instagram-style implicit behavioral model) is a Phase 3+ capability requiring real usage volume and is explicitly out of scope for v1.

If asked to cut corners on any of the above to ship faster, flag it back to the founder rather than silently complying.

1. Product Vision

Trylo is a curated fashion marketplace for Pakistan, positioned as a more trustworthy, more personalized alternative to Daraz — not a full horizontal marketplace, but a "digital department store" for fashion specifically.

The core differentiation is sequencing, not features:

1. Curated swipe-based discovery (find your taste fast)
2. Behavioral memory (get better at predicting taste over time)
3. Virtual try-on (reduce purchase hesitation at the point of decision)
4. A trust architecture that directly inverts Daraz's most-complained-about failure points (see Section 9)

Geographic scope for v1: Lahore-based sellers and buyers. Hafizabad is a planned future operations base, not a launch market (sellers there are not yet digitally ready, per founder's on-the-

ground assessment).

Audience: Both women's and men's fashion, in one unified app with a persistent department toggle (see Section 5). Women's is the primary focus; men's categories exist alongside, not as an afterthought, but should not block or dilute the women's experience.

2. The Business Model

- **Trylo does not hold inventory.** Verified third-party sellers (think: individual boutiques, designers, small fashion businesses based in Lahore) list and fulfill their own products.
- **Trylo owns the discovery experience and the customer relationship/memory** — not the product itself. This is the same model as a curated department store: many brands, one trusted shopping experience.
- **Revenue model** (not yet finalized — flag to founder, don't assume): likely commission-per-sale, possibly featured-placement fees for sellers later. Do not hard-code a specific commission %; make it configurable.

Two-Gate Seller Trust Architecture

This is foundational, not a nice-to-have — it directly differentiates Trylo from Daraz's anonymous, low-accountability seller pool.

- **Gate 1 — Verification (one-time, onboarding):** CNIC verification, business proof/registration if available, a real physical or business address, a phone number verified via OTP. A seller cannot list products until Gate 1 is cleared.
 - **Gate 2 — Ongoing Reputation (continuous):** Customer ratings, response time to messages, return/refund rate, dispute history. This should directly affect a seller's ranking in search/discovery results — sellers with worse Gate 2 metrics surface less prominently, without being delisted outright unless egregious.
 - **Full transparency by design:** the seller's name, rating, and sales count are shown to the customer at every touchpoint (swipe card, product page, results) — never hidden. This is a deliberate trust signal copied from how real boutique-style platforms operate, and a direct contrast to Daraz's often-opaque seller identity.
-

3. Department Structure (Men / Women)

A persistent toggle at the top of the app switches between **Women** and **Men** departments. This is the same pattern used by Myntra, Ajio, and most mature fashion e-commerce apps — not a novel UX choice, a proven one.

- Switching departments filters: category browse chips, search autocomplete suggestions, swipe queue, results, and "more like this" recommendations. There should be **zero cross-department leakage** — a men's department search must never surface women's products and vice versa.

- Every product is tagged with a `department` field (`men | women`) at the data layer. This is not inferred at query time — it's set explicitly when a seller lists a product, with sensible defaults based on category (e.g., "sherwani" defaults to men, "lehenga" defaults to women) but is always overridable.
 - Every product detail view explicitly states "For Men" or "For Women" in its description — copying the disambiguation pattern used by Daraz/Amazon/Myntra precisely because founder testing surfaced real confusion when this wasn't explicit.
 - Shared categories (e.g., "shoes") exist in both departments with department-specific sub-styles (e.g., Women: heels, khussa, kolhapuri flats. Men: Oxfords, loafers, peshawari chappal, sneakers).
-

4. Category Taxonomy

The taxonomy must support **two levels**: a broad root category (e.g., "abaya," "shirt") and specific sub-styles within it (e.g., "formal abaya," "open-front abaya"). This is what powers both autocomplete and the discovery-swipe flow described in Section 6.

Women's root categories (v1 scope):

lawn, kameez, kurta, suit (3-piece/2-piece), abaya, bridal (lehenga, gharara, saree, maxi, walima gown), lehenga, gharara, saree, maxi, anarkali, frock, shawl, party-wear, formal-wear, shoes (heels, khussa, kolhapuri)

Men's root categories (v1 scope):

shirt (formal, casual, printed, linen, polo), shalwar kameez (plain, wash & wear, kurta shalwar, embroidered), waistcoat, sherwani / prince coat, trousers (dress, chino, cotton), shoes (formal/Oxford, loafers, peshawari/norozi chappal, sneakers)

Each root category must have at least 4-6 named sub-styles for autocomplete to be useful. This list will grow as real sellers onboard with their own product types — the schema should not hard-code this list; it should be a manageable, editable taxonomy table, not enum constants in code.

Search requirements:

- **Typo tolerance is mandatory.** Real user testing surfaced this as a hard failure: a user searching "abayia" (misspelled "abaya") returned an unrelated random product instead of abaya results. Implement fuzzy matching (edit-distance / Levenshtein, or a proper search engine's built-in fuzzy matching — Elasticsearch, Typesense, Algolia, or Postgres `pg_trgm` are all viable) so close misspellings still resolve correctly. When a fuzzy correction is made, tell the user explicitly ("Showing results for 'abaya' — closest match to what you typed") rather than silently correcting.
- **Category-faithful results are mandatory.** If a user searches "abaya," every single result — in swiping and in final results — must be an abaya. Mixing in unrelated categories due to a search-matching fallback bug is a confirmed real failure mode from testing; the production search must guarantee category fidelity, with a typo-corrected category still counting as

faithful.

- **Vague vs. specific search handling:** If a user types a broad root term ("abaya"), the discovery flow should show one representative item per sub-style so they can pick a direction (see Section 6, Discovery Mode). If they type or autocomplete-select a specific sub-style ("formal abaya"), skip straight to a varied swipe set within that sub-style.

5. Core User Flow

This is the validated interaction sequence from prototype testing — preserve this sequencing exactly, it is the core product mechanic, not an implementation detail.

```
Search (text input + autocomplete)
↓
Swipe Phase 1 – "Benchmark" (3 cards, weighted 2x)
  ↓ [system infers a taste hypothesis, shows a brief "read" of it]
Swipe Phase 2 – "Confirmation" (2 cards, weighted 1x, refined based on Phase 1)
↓
Results screen (4-6 best matches, ranked, with a plain-language explanation of WHY these were chosen)
↓
Product detail page (full info, seller info, "More like this" rail)
↓
Try-On (ONLY available here – never during swiping)
↓
Order intent → seller contact (WhatsApp handoff for v1, or in-app checkout for later phases)
```

Swipe mechanics:

- **Three gesture directions**, not two: swipe right = strong positive (full weight), swipe left = negative, swipe up = "medium/maybe" — a soft positive signal at half weight. This third option matters: forcing a binary like/dislike discards real signal when users are ambivalent, which was a friction point in testing.
 - **First-time tutorial:** on a user's first swipe screen each session, show a brief overlay explaining the three gestures (arrows + labels: right=love/save it, left=skip/not for me, up=maybe/unsure but ok) before they start swiping.
 - **Skip-to-results option:** a visible "skip" control (top-right of swipe screen) lets a user bypass personalized swiping and jump straight to popular/default results in that category. This must not be hidden — repeated forced swiping was identified as an annoyance risk in user feedback. When skipped, the results screen should honestly say results are "popular in [category]" rather than implying personalization that didn't happen.
 - **Discovery Mode for vague searches:** when a search term is a broad category root rather than a specific sub-style, Phase 1 should show one product per distinct sub-style (e.g., search "bridal" → show one lehenga, one gharara, one saree as the three benchmark cards) so the swipe itself becomes the mechanism for narrowing down what she actually means by "bridal." The system should not ask "which category" via a menu — letting her react to real products IS the category-narrowing mechanism.
-

6. The Recommendation / "Memory" Logic

This is the algorithmic core of the product and needs the most engineering care to get right, even at MVP scope.

Two-layer model: surface attributes + taste families

Layer 1 — Surface attributes (tagged per product at the data layer):

- **pattern:** plain | printed | embroidered | embellished
- **tone:** warm | cool | neutral
- **brightness:** light | mid | dark
- **saturation:** muted | bright
- **formality:** casual | semi-formal | formal
- **coverage:** modest | full
- **cut:** fitted | flowy | straight
- **silhouette:** category-specific (kurta, suit, abaya, lehenga, gharara, saree, maxi, anarkali, frock, shawl-set, shirt, mens-suit, waistcoat, sherwani, trouser, etc.)

Layer 2 — Derived taste families (computed from swipe history, NOT tagged manually): This is the more important layer and was validated as the key emotional "aha" moment in testing. A real example: a tester rejected red and green during swiping, picked light colors — but in the final results, chose a BLACK item over a white one. The lesson: she wasn't responding to "light vs dark," she was responding to **plain/restrained vs busy/loud**, a deeper principle that spans color depth.

Compute derived signals such as:

- **restraint_score** = (plain swipes) – (printed + embroidered + embellished swipes).
Positive = prefers uncluttered designs regardless of color depth.
- **palette_muted_score** = (muted saturation swipes) – (bright saturation swipes).
- **Dominant tone / formality / silhouette** = whichever value accumulates the most positive weighted signal.

Ranking logic for "next cards" and "results":

- Score each candidate product against the derived taste families FIRST (restraint, palette_muted), THEN against surface attributes (tone, formality, silhouette) as secondary signal.
- When a taste signal is strong/unambiguous, weight matching attributes more heavily and mismatches more harshly — i.e., the more confident the system is, the tighter (more clustered) the results should be. When the signal is weak/mixed, keep results more exploratory/varied.
- Left-swiped (disliked) products must never reappear in the same session.
- Already-shown products (whether liked or maybe'd) should be down-ranked in subsequent

recommendations within the same session so Phase 2 brings genuinely fresh picks, not repeats.

Weighting by swipe phase and gesture:

- Phase 1 (benchmark) swipes: 2x weight
- Phase 2 (confirmation) swipes: 1x weight
- Up-swipe ("maybe"): half the weight of whatever phase it's in, and counts as a POSITIVE signal (not negative) — this was a confirmed implementation bug to avoid: treating "maybe" as a dislike is wrong, it should nudge toward the attributes at reduced strength.

Variety enforcement (important, confirmed failure mode if skipped):

When there is little or no taste signal yet (e.g., user skipped swiping entirely), do NOT fall back to raw catalog/database insertion order for results — this was a confirmed bug that caused results to cluster on whichever 1-2 sellers happened to be listed first. Instead, actively diversify: when ranking is undifferentiated, maximize spread across different sellers (weight this most heavily) and different sub-styles/patterns, with some randomization in the starting point so repeated "skip" sessions don't show an identical default set every time.

"More Like This" — a distinct, separate recommendation surface

On the product detail page, a horizontally-scrolling "More like this" rail shows items related to the SPECIFIC product being viewed — not the user's overall swipe-derived taste profile. This is intentionally a different signal: same seller (weighted highest), same silhouette, same pattern family, same tone, same broad category, roughly similar price band. Each card should show a one-line reason ("same seller," "similar cut," etc.) — real e-commerce platforms do this (Amazon's "customers who viewed this also viewed"), and it serves a different purpose than the personalized results grid.

Explicit + future implicit signals

v1 should rely entirely on **explicit signals**: swipes (right/left/up), product opens, try-on usage, order intent clicks. Architect the data model so **implicit signals** (dwell time, scroll-back behavior, zoom interactions) can be added later as additional weighted inputs once usage volume is high enough for them to be statistically meaningful (this is a TikTok/Instagram-style pattern — implicit signals need volume to not be noisy). Do not attempt to build implicit-signal modeling for v1; just don't architecturally block it.

7. Virtual Try-On — Honest Technical Approach

Do not build a custom try-on model in-house for v1 or v2. This is a mature, hard ML problem (garment warping + diffusion-based image synthesis) that specialist companies have spent years and significant capital on. Building from scratch is not a reasonable MVP investment.

Recommended approach: integrate a third-party virtual try-on API.

- Candidates to evaluate at build time (pricing/availability changes — verify current state):

Aiuta, FASHN AI, or similar commercial virtual try-on APIs designed for e-commerce integration.

- Integration shape: user uploads a photo (or selects a stored one), selects a product, the API returns a composited try-on image. Trylo's job is the UI/UX wrapper and the privacy handling around it — not the underlying garment-rendering model.
- **Try-on must only be exposed on the product detail page**, never during the swipe/discovery phase. This was an explicit founder correction during prototyping — try-on belongs at the point of serious consideration, not the point of casual browsing, both for UX reasons (it would slow swiping down) and a clear business reason (you don't want to burn try-on API costs on items being casually browsed).

Privacy requirements (non-negotiable, validate before any photo leaves the device or hits a server):

- User's photo must never be shown to or stored by sellers.
- User's photo should not be retained beyond the active session unless the user explicitly opts in to saving it for convenience (e.g., "remember my photo for faster try-on next time" as an explicit, unchecked-by-default toggle).
- State this plainly in-product near the try-on button, not buried in a ToS page — testing showed this is a real point of hesitation, particularly for more conservative users (per founder's note, this came up as a specific concern from a female tester).

Fallback / honest framing while no real try-on integration exists yet: if try-on isn't live yet at any build stage, the UI should say so plainly rather than mocking a fake result — do not ship a "fake" try-on that pretends to be real.

8. Product Photography — Legal & Practical Requirements

- **All product images must be sourced from real, onboarded sellers who own the rights to them.** Never scrape, screenshot, or otherwise pull images from external websites, social media, or competitor listings — this is a copyright/legal requirement, full stop.
 - Build the seller onboarding flow to require photo upload as part of listing a product (multiple angles minimum: front, optionally back/detail/fabric-close-up).
 - The product data model's image field should support multiple photos per product (gallery), not just one.
 - Until a seller's real photos exist, do not silently substitute a stock/generic image that could mislead a buyer about what they're getting — either show an honest "photos coming soon" placeholder or block the listing from going live until real photos are uploaded.
-

9. Post-Purchase Trust Features (from Daraz failure-mode research)

Research into real Daraz customer and seller reviews surfaced that the majority of trust-destroying failures happen AFTER purchase, not during discovery — and this is an area with almost no good AI/product solutions currently in market. These should be treated as core differentiators, not nice-to-haves, and should be planned into the roadmap even if not all are in the v1 cut:

1. **Packaging-proof photo requirement.** Before sealing a package, the seller must photograph the packed contents (the actual item(s) about to be shipped). This photo is attached to the order and visible to the buyer. This directly addresses one of the most common Daraz complaints: "wrong item received" / "package arrived empty" disputes where there's no way to verify what was actually shipped. This is cheap to build (just a mandatory photo-upload step in the seller fulfillment flow) and high-trust-value.
 2. **Transparent, trackable refund/payment status.** Build a visible status timeline (like a bank transaction or shipment tracker) for both buyer refunds and seller payouts — not an opaque "processing" state. Daraz's worst-reviewed failure pattern is refunds/payments being silently stuck for months with no visibility. Even a simple status-stage tracker (Requested → Under Review → Approved → Paid, with timestamps) is a major trust improvement over silence.
 3. **(Later phase) AI-assisted mismatch verification.** If a buyer reports "wrong item received," allow them to photograph what arrived and have a model flag an obvious visual mismatch against the listing photo (color, print, fabric texture) as supporting evidence in a dispute — this is a genuine, scoped use of AI in the post-purchase flow, distinct from and much simpler than try-on. Not required for v1, but architect dispute records to support attaching photos so this can be added later without a data model migration.
 4. **Support tooling should optimize for resolution speed, not deflection.** If/when an AI support layer is built, its success metric should explicitly be "issue resolved" not "conversation ended" — Daraz's existing AI chat support is widely reported by users as designed to deflect rather than resolve, and inverting this is a real, achievable differentiator.
-

10. Feedback & Analytics Instrumentation

Founder testing methodology relies on structured, in-session data capture rather than only post-hoc surveys. Build this into the real product from day one as first-class analytics events, not bolted on later:

In-flow micro-feedback ("reaction pulses"): at specific moments — after the Phase 1 benchmark swipes, after results are shown, after a try-on is used — surface a lightweight, dismissible 2-3 option reaction prompt (e.g., after results: "Personal / So-so / Random"). Log the response with a timestamp and the session/user context.

Session-level structured exit survey (optional, skippable): after a session, a short survey — a 1-5 "did this feel personalized" scale, multiple choice on willingness to upload a real photo for try-on, multiple choice on purchase trust, and an optional free-text field (must support both Urdu and English input).

Events to track at minimum:

- search terms (raw input + resolved category, including whether a fuzzy/typo correction occurred)
- department switches
- swipe events (product id, direction, phase, session)
- skip-to-results usage
- products opened, "more like this" clicks
- try-on usage
- order-intent clicks (i.e., reached seller contact / checkout)
- all reaction-pulse and exit-survey responses

This data should be queryable by the founder (a simple internal dashboard is fine for v1 — does not need to be polished) to review patterns across test sessions, not just individual sessions.

11. Data Model (high-level entities)

This is not a full schema — it's the entity list and key relationships an engineer needs to design the real schema correctly.

- **User** — buyer or seller (or both), auth, profile, department preference (last-used, not exclusive), saved try-on photo (optional, opt-in only)
 - **Seller** — linked to User, Gate 1 verification status + documents, Gate 2 reputation metrics (rating, response time, return rate, dispute count), business name, location
 - **Product** — belongs to Seller, department (men/women), root category, sub-style, attributes (pattern/tone/brightness/saturation/formality/coverage/cut/silhouette), price, image gallery, stock/size info, status (draft/live/suspended)
 - **Taxonomy** — root categories and their sub-styles, editable (not hardcoded), department-tagged
 - **SwipeEvent** — user/session, product, direction (left/right/up), phase, timestamp, computed weight
 - **Session** — groups swipe events, search terms, department, and feedback responses together for analytics
 - **Order** — buyer, seller, product(s), status, packaging-proof photo, payment status timeline
 - **Dispute** — order reference, type (wrong item, damaged, not delivered, refund delay, etc.), evidence photos, status timeline
 - **Review/Rating** — buyer, seller, order reference, score, text (feeds Gate 2 reputation)
 - **FeedbackEvent** — reaction pulses and exit survey responses, linked to session
-

12. Suggested Tech Stack (recommendation, not mandate — confirm with engineering team)

- **Frontend:** React (web) with a mobile-first responsive design; consider React Native later if a native app is needed, but a well-built mobile web app is a reasonable v1 given founder's solo non-technical status and budget constraints.
 - **Backend:** Node.js/TypeScript or Python (FastAPI/Django) — either is fine; pick based on whoever is actually building this.
 - **Database:** PostgreSQL for relational data (users, products, orders, disputes). Consider a dedicated search service (Typesense, Meilisearch, or Postgres full-text + `pg_trgm` for fuzzy match) for the product search/autocomplete layer specifically — general-purpose SQL LIKE queries will not deliver acceptable fuzzy-search UX at scale.
 - **Image storage:** S3-compatible object storage (S3, Cloudflare R2, or similar) for product photos and gallery images.
 - **Try-on integration:** third-party API (see Section 7) — do not build in-house.
 - **Analytics:** can start as simple structured event logging into Postgres; doesn't need a dedicated analytics platform for v1 given expected volume.
-

13. Suggested MVP Build Order

1. **Seller onboarding + Gate 1 verification flow** (nothing works without real sellers and real products)
2. **Product listing flow** (including mandatory photo upload, taxonomy tagging, department tagging)
3. **Search + taxonomy + fuzzy matching**
4. **Swipe discovery flow** (3+2 phases, three gestures, tutorial overlay, skip option)
5. **Recommendation/scoring engine** (surface attributes first, taste-family layer can follow once basic ranking works)
6. **Results + product detail + "more like this"**
7. **Order intent → seller contact handoff** (WhatsApp deep link is an acceptable v1 — does not need full in-app checkout/payments immediately)
8. **Feedback instrumentation** (build this alongside everything above, not after — it's how the founder will know what to fix next)
9. **Gate 2 reputation system** (ratings, response time tracking) — needed once real orders start flowing
10. **Packaging-proof photo step** in seller fulfillment flow
11. **Try-on API integration** (can be sequenced later — it is the third pillar by design, not the first thing users see)

12. Refund/payment status tracker

Department toggle (Men/Women) and the men's catalog should be present from the start, not bolted on later — but do not let men's-side completeness block the women's-first focus; women's is the primary launch wedge per founder's market positioning.

14. What NOT To Do

- Do not scrape or otherwise source product images from anywhere other than real onboarded sellers.
 - Do not build or promise "proportional pixel scaling" or any custom-built body-accurate try-on model. Integrate, don't build, for try-on.
 - Do not let search fall back to raw/unfiltered catalog results on a typo or no-match — always either fuzzy-correct with a visible note, or clearly say no results were found.
 - Do not let an empty/weak taste signal fall back to raw database insertion order for results — actively enforce variety (seller + style diversity) when there's no real personalization signal yet.
 - Do not expose try-on during the swipe/discovery phase — it belongs only on the product detail page.
 - Do not treat the "maybe" (up-swipe) gesture as a negative signal — it is a reduced-weight positive signal.
 - Do not silently fake try-on results if the real integration isn't live yet — say so honestly in the UI.
 - Do not hide seller identity, rating, or sales count anywhere in the buyer-facing flow — full transparency is a core trust differentiator, not optional.
-

Appendix A — Existing Prototype Reference

A working single-file HTML/JS interactive prototype (no backend, illustrated SVG garments standing in for real photos) has been built and iteratively tested with real users in Lahore. It demonstrates and validates:

- The full search → swipe (3+2 phases) → results → product → try-on flow
- The three-gesture swipe mechanic (left/right/up) with the first-time tutorial overlay
- Department toggle (Men/Women) with full category, search, and results isolation
- Taxonomy-driven autocomplete with typo tolerance
- The taste-family scoring logic described in Section 6
- The in-flow reaction-pulse + exit-survey feedback system described in Section 10
- "More like this" product-level recommendations

This prototype should be treated as a UX/interaction and logic reference only — not a codebase to extend or port directly. Its illustrated SVG garments and lack of a real backend, database, or auth system mean it cannot evolve into the production app; the production app should be built fresh against this specification, using the prototype to resolve any ambiguity about exact intended behavior (e.g., "what should the swipe tutorial actually say" or "what does the results explanation text sound like").

Appendix B — Key Validated User Insights (carry these into design decisions)

- Real testers respond to a **deeper "taste family" principle** (e.g., plain/restrained vs. busy/ornate) more than to surface attributes like literal color. A tester who rejected red/green and picked light colors during swiping still chose a black item over white in final results — because she was responding to "plain," not "light." Get this right; it's the emotional core of why the memory feature would feel "magic" rather than generic.
- Users want explicit acknowledgment when the system corrects their input (typos) rather than silent correction.
- Forced binary swiping (only like/dislike) creates friction for genuinely ambivalent reactions — the "maybe" gesture exists specifically to solve this.
- Repeated near-identical results after skipping personalization is a credibility killer — variety must be actively engineered, not assumed to emerge naturally from "more data later."
- At least one tester expressed real hesitation about uploading a personal photo for try-on — privacy framing at the point of asking matters, not just in a privacy policy.
- Mixing departments without explicit labeling caused real confusion in testing — "For Men"/"For Women" labels are not cosmetic, they were added in direct response to confirmed user confusion.